Reviewer Pack — Minimal Order of Galois Covers and Communication Rank Growth

Entry 30a56bd45595 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 07:16:01 UTC

Minimal Order of Galois Covers and Communication Rank Growth

Entry ID: `30a56bd45595` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 07:16:01 UTC

1. Conjecture

Field A (mathematical branch): Galois Theory **Field B** (complexity object): Communication Complexity

Statement:

> The minimal order of a Galois cover for a given graph G is linearly correlated with the growth rate of its communication complexity, such that the maximum edge connectivity κ(G) is Θ(κ(G)^2 log n)

Rationale (proposer's reasoning):

> Galois theory provides a framework to study the structure of fields and their extensions. The minimal order of a Galois cover can be seen as a measure of the field's complexity. Communication complexity, on the other hand, measures the amount of communication needed for two parties to compute a function. A linear correlation between these invariants may reveal a deep connection between the algebraic structure of graphs and the information-theoretic properties of their communication.

Taxonomy category: `Galois_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5573bcc974183a6f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all graphs G with 1 ≤ n ≤ 40 vertices, the growth rate of communication complexity satisfies κ(G)^2 log n ≥ 0.5 * metric_mean where metric_mean is the average growth rate across all seeds and support_fraction ≥ 0.8.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_graph(n):
    if n <= 1:
        return [], 0

    nodes = list(range(n))
    edges = []

    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                edges.append((i, j))

    return (nodes, edges), n

def gaussian_elimination(A, b):
    n = len(b)
    A_b = [A[i] + [b[i]] for i in range(n)]

    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A_b[j][i]) > abs(A_b[max_row][i]):
                max_row = j

        A_b[i], A_b[max_row] = A_b[max_row], A_b[i]

        for j in range(i+1, n):
            factor = A_b[j][i] / A_b[i][i]
            A_b[j] = [A_b[j][k] - factor * A_b[i][k] for k in range(n + 1)]

    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (A_b[i][-1] - sum(A_b[i][j] * x[j] for j in range(i+1, n))) / A_b[i][i]
```

```
    return x

def max_edge_connectivity(graph):
    nodes, edges = graph
    n = len(nodes)
    if n <= 1:
        return 0

    max_kappa = 0
    for node in nodes:
        neighbors = [neighbor for neighbor in nodes if (node, neighbor) in edges or (neighbor, noc
        kappa = len(neighbors)
        if kappa > max_kappa:
            max_kappa = kappa

    return max_kappa

def communication_rank(graph):
    nodes, edges = graph
    n = len(nodes)

    A = [[0] * n for _ in range(n)]
    b = [0] * n

    for u, v in edges:
        A[u][v] += 1
        A[v][u] += 1
        b[u] += 1
        b[v] += 1

    x = gaussian_elimination(A, b)

    return sum(x[i] for i in range(n))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    instances_tested = 0
    total_growth_rate = Fraction(0)
    max_n = 0

    for n in n_values:
        graph, n = generate_random_graph(n)
        kappa = max_edge_connectivity(graph)
        growth_rate = communication_rank(graph) / (kappa ** 2 * math.log(n))

        instances_tested += 1
        total_growth_rate += Fraction(growth_rate).limit_denominator()
        if n > max_n:
            max_n = n

    metric_mean = total_growth_rate / instances_tested
    conjecture_holds = all(growth_rate >= 0.5 * metric_mean for growth_rate in [communication_rank
```

```
    return {
        "metric_name": "growth_rate",
        "metric_value": float(metric_mean),
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(3, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_growth_rate = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_growth_rate} std=0.0 support_fraction={support_fracti
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_growth_rate} std=0.0 support_fraction={support_fracti
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ae62ac6.py", line 122, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ae62ac6.py", line 97, in run_trial
    growth_rate = communication_rank(graph) / (kappa ** 2 * math.log(n))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ae62ac6.py", line 82, in communication
    x = gaussian_elimination(A, b)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ae62ac6.py", line 45, in gaussian_elim
    factor = A_b[j][i] / A_b[i][i]
             ~~~~~~~~~~^~~~~~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

The test crashed due to a division by zero error, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the division by zero error in the code. Once the issue is resolved, rerun the test to verify if the conjecture's support conditions are met.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13310
2	propose	ollama_remote	glm4:latest	0	0	15781
3	preregistration	ollama_remote	glm4:latest	0	0	20382
4	novelty	ollama_remote	glm4:latest	0	0	8726
5	novelty	ollama_remote	glm4:latest	0	0	8302
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19285
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11252
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15706
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12815
10	judge	ollama_remote	glm4:latest	0	0	9943

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 135501 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/30a56bd45595.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/30a56bd45595.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/30a56bd45595.tar.gz` (if generated)